

Chương 4

Chuyển hướng & Pipe — điều khiển luồng dữ liệu

(stdin, stdout, stderr)

Giáo trình Linux cho người mới bắt đầu

Mục lục

1	Mục tiêu học tập	2
2	Câu chuyện nền: một lệnh có “ba cái ống”	2
2.1	Vì sao phải tách stderr khỏi stdout?	3
3	Chuyển hướng đầu ra (stdout): >, >>, 1>	4
4	Chuyển hướng lỗi (stderr): 2>, 2>>	4
4.1	Gộp lỗi vào chung đầu ra: 2>&1	5
4.2	THỨ TỰ rất quan trọng	5
4.3	Cú pháp gọn &>	5
5	/dev/null — cái “hố đen” nuốt mọi thứ	6
6	Chuyển hướng đầu vào (stdin): <	6
7	Pipe — nối các lệnh thành dây chuyền	7
7.1	Pipe khác Redirect như thế nào?	7
7.2	Vài lệnh hay đi cùng pipe	7
8	THỰC HÀNH NGAY (Lab chính)	8
8.1	Bước 1 — Tạo sân tập và vào đó	8
8.2	Bước 2 — stdout chảy vào file	9
8.3	Bước 3 — stderr chảy vào file (lỗi KHÔNG hiện ra màn hình)	9
8.4	Bước 4 — Nuốt lỗi bằng /dev/null	9
8.5	Bước 5 — Pipe để đếm	9
8.6	Bước 6 — Pipe vào sort	10
8.7	Bước 7 — Pipe vào grep để lọc	10
8.8	Bước 8 — Gộp kết quả và lỗi vào cùng một file	10
8.9	Bước 9 — Dọn dẹp	10
9	Bảng tóm tắt ký hiệu (in ra dán bàn)	11
10	Góc Senior (vì sao, production, cạm bẫy)	11
11	Hiểu làm thường gặp (đọc kỹ để khỏi sai)	13
12	Thẻ ghi nhớ (Flashcards)	14
13	Kế hoạch ôn ngắt quãng (Spaced Repetition)	14
14	Tóm tắt 1 trang	15
15	Bài tập mở rộng (CÓ đáp án)	15
15.1	Đáp án Bài tập mở rộng	15
15.2	Đáp án phần “Tự kiểm tra”	17

Ở Chương 3 bạn đã thấy dấu `>` và `>>` ghi kết quả ra file. Chương này giải thích “bộ máy” đằng sau: dữ liệu chảy vào và ra khỏi một lệnh như thế nào, và làm sao bạn điều khiển được dòng chảy đó. Đây là một trong những kỹ năng “biến người dùng thường thành người dùng có nghề”.

Lưu ý môi trường

Trên macOS hiện nay, shell mặc định là **zsh** (không phải bash). Mọi ví dụ trong chương chạy tốt trên zsh. Một vài chỗ ta sẽ chỉ rõ khi zsh và bash hành xử khác nhau, để bạn không bối rối khi thử trên máy mình.

1 Mục tiêu học tập

Học xong chương này, bạn sẽ:

1. Hiểu **ba luồng chuẩn**: **stdin** (đầu vào), **stdout** (đầu ra), **stderr** (lỗi) — và vì sao lỗi lại tách riêng.
2. **Chuyển hướng đầu ra** ra file: `>`, `>>`, `1>`.
3. **Chuyển hướng lỗi** ra file: `2>`, `2>>`, và gộp lỗi vào chung đầu ra bằng `2>&1` hoặc `&>`.
4. Dùng `/dev/null` (“hố đen”) để vứt bỏ thứ bạn không cần.
5. **Chuyển hướng đầu vào** từ file bằng `<`.
6. Dùng **pipe** `|` để nối nhiều lệnh thành “dây chuyền xử lý”.
7. Phân biệt rõ **pipe** (nối lệnh với lệnh) và **redirect** (nối lệnh với file).
8. Biết các cạm bẫy production: thứ tự `2>&1`, mã thoát của pipe, vì sao tách lỗi quan trọng khi viết log.

2 Câu chuyện nền: một lệnh có “ba cái ống”

Hãy tưởng tượng mỗi lệnh trên Terminal giống một **người công nhân** có ba cái ống nối ra ngoài:

- **stdin** (standard input — *đầu vào chuẩn*, mang **số hiệu 0**): cái **tai nghe** / **bàn phím**. Đây là nơi dữ liệu **đi vào** lệnh. Mặc định, nó nối tới bàn phím của bạn.
- **stdout** (standard output — *đầu ra chuẩn*, mang **số hiệu 1**): cái **miệng** để nói ra **kết quả**. Mặc định, nó nối ra màn hình.
- **stderr** (standard error — *lỗi chuẩn*, mang **số hiệu 2**): cái **còi báo lỗi** riêng. Khi có trục trặc, lệnh kêu qua còi này. Mặc định nó **cũng ra màn hình**, nhưng là **một kênh tách biệt** với stdout.

Định nghĩa nhanh — “luồng” (stream)

là một dòng dữ liệu chảy liên tục (như nước chảy trong ống), không phải một file cố định. Ba luồng trên là ba “ống nước” mặc định mà hệ điều hành gắn sẵn cho mọi lệnh.

Định nghĩa nhanh — “số hiệu” (file descriptor)

mỗi ống có một con số để gọi tên. 0 = stdin, 1 = stdout, 2 = stderr. Khi bạn gõ `2>` nghĩa là “lấy ống số 2 (lỗi) và chuyển hướng nó”. Tên tiếng Anh có chữ *file* vì với Unix, **mọi thứ — kể cả ống dữ liệu, màn hình, bàn phím — đều được đối xử như một dạng file**. Bạn chưa cần bận tâm điều đó; giờ chỉ cần nhớ ba con số 0/1/2.

Định nghĩa nhanh — “tiến trình” và “tiến trình con” (process / subprocess)

mỗi lệnh đang chạy là một **tiến trình**. Khi shell tạo thêm một bản sao của chính nó để chạy một phần việc riêng, bản sao đó gọi là **tiến trình con**. Vì nó là một bản sao riêng biệt, các thay đổi bên trong nó (ví dụ đặt một biến) **có thể không truyền ngược ra ngoài**. Ta sẽ gặp lại khái niệm này ở Mục 10 khi nói về pipe.

Chính xác hơn (không bắt buộc nhớ)

cả ba luồng mặc định cùng trở tới **cửa sổ Terminal đang chạy** (gọi là *tty*), nên ta thấy cả kết quả lẫn lỗi trên màn hình, và gõ bàn phím thì lệnh nhận được. “Màn hình / bàn phím” chỉ là cách hình dung trực quan. Khi lệnh chạy nền (cron, dịch vụ chạy ngầm) thì “màn hình” có thể không tồn tại — lúc đó việc tách hai luồng để ghi log càng quan trọng (xem Mục 10).

2.1 Vì sao phải tách stderr khỏi stdout?

Tưởng tượng bạn bảo công nhân: “đếm giúp tôi số hộp hàng, rồi tôi mang con số đó đi tính tiền”. Nếu lúc đếm có sự cố (“hộp số 7 bị rách!”), bạn **không muốn** câu cảnh báo đó lẫn vào con số kết quả — vì bạn sắp mang **kết quả** đi tính tiền tự động.

Đó chính là lý do Unix tách **kết quả** (stdout) và **thông báo lỗi** (stderr) thành hai ống riêng:

- Bạn có thể lấy **stdout** đẩy sang lệnh khác xử lý tiếp.
- Còn **stderr** thì ghi riêng vào file log lỗi, hoặc hiện lên màn hình cho người vận hành thấy.

Hai luồng độc lập = không lẫn lộn. Đây là nền tảng của mọi automation (tự động hóa) sau này.

Tự kiểm tra 2

- a) `stdin`, `stdout`, `stderr` mang số hiệu lần lượt là mấy?
b) Mặc định `stderr` hiện ra đâu? Nó có giống `stdout` không?
(Đáp án ở cuối chương.)

3 Chuyển hướng đầu ra (`stdout`): `>`, `>>`, `1>`

Bạn đã gặp ở Chương 3. Nhắc lại ngắn gọn:

Ký hiệu	Ý nghĩa
<code>></code>	Ghi <code>stdout</code> ra file, ghi đè (xóa nội dung cũ rồi viết mới).
<code>>></code>	Ghi <code>stdout</code> ra file, nối thêm vào cuối (giữ nội dung cũ).
<code>1></code>	Giống hệt <code>></code> — chỉ là viết rõ “ông số 1”.

Vì `stdout` mang số hiệu 1, nên `>` thực ra là dạng viết tắt của `1>`. Người ta hiếm khi gõ `1>` vì `>` đã đủ; nhưng hiểu điều này giúp bạn đọc được cú pháp `2>` sắp tới.

```
echo "dong mot" > file.txt      # tao file, ghi de
echo "dong hai" >> file.txt     # noi them dong hai
```

Cạm bẫy

`>` **xóa sạch** nội dung cũ ngay khi bạn nhấn Enter. Gõ nhầm tên file quan trọng = mất dữ liệu. Khi chưa chắc, hãy dùng `>>` (nối thêm) cho an toàn hơn.

4 Chuyển hướng lỗi (`stderr`): `2>`, `2>>`

Đây là điểm mới quan trọng nhất của chương.

- `2>` — lấy **ông số 2 (lỗi)** và ghi ra file (ghi đè).
- `2>>` — ghi lỗi ra file, nối thêm.

Mấu chốt: `>` **chỉ** chuyển hướng `stdout`. Nó **không** bắt được lỗi. Muốn bắt lỗi, bạn **phải** dùng `2>`.

```
ls khong-co-thu-muc-nay 2> loi.txt
```

Lệnh trên cố liệt kê một thư mục không tồn tại. Thông báo lỗi (`stderr`) sẽ **không hiện trên màn hình** mà chui vào `loi.txt`.

4.1 Gộp lỗi vào chung đầu ra: 2>&1

Đôi khi bạn muốn **cả kết quả lẫn lỗi vào cùng một file**. Cú pháp:

```
lenh > out.txt 2>&1
```

Đọc là: “đưa stdout vào out.txt (> out.txt), rồi đưa stderr **đi theo nơi stdout đang trở tới** (2>&1)”.

Giải mã 2>&1

2> = chuyển hướng ống số 2; &1 = “tới chỗ mà ống số 1 đang trở”. Dấu & ở đây **báo cho máy biết: con số 1 đứng sau là MỘT CÁI ỐNG (file descriptor) đang tồn tại, chứ không phải tên file tên “1”**. Nếu thiếu dấu &, máy sẽ hiểu 1 là tên file và tạo ra một file tên “1”.

4.2 THỨ TỰ rất quan trọng

Hai dòng dưới đây **trông giống nhau nhưng kết quả khác hẳn**:

```
lenh > out.txt 2>&1      # DUNG: cả stdout và stderr vào out.txt
lenh 2>&1 > out.txt      # SAI Y DÒ: stderr vẫn ra màn hình
```

Vì sao? Shell đọc **từ trái sang phải**:

- Dòng đúng: trước hết stdout trở vào file; sau đó 2>&1 bảo stderr “đi theo stdout” — lúc này stdout đã ở file, nên lỗi cũng vào file.
- Dòng sai: 2>&1 chạy **trước**, lúc đó stdout vẫn đang trở ra **màn hình**, nên stderr cũng trở ra màn hình. Sau đó > out.txt mới đổi stdout sang file — nhưng stderr đã “chốt” ở màn hình rồi, không đổi theo.

Hãy nhớ câu thần chú: “**file trước, gộp sau**” — > out.txt 2>&1.

4.3 Cú pháp gọn &>

zsh và bash hỗ trợ lỗi viết tắt:

```
lenh &> out.txt          # gộp cả stdout VÀ stderr vào out.txt (ghi de)
lenh &>> out.txt          # gộp cả hai, nơi thêm
```

&> tiện và dễ nhớ, nhưng nó là **mở rộng của zsh/bash**, không phải chuẩn POSIX.

Cảnh báo dành cho script

Trên shell sh/dash tối giản (thường là /bin/sh trên server), &> **không** được hiểu là “gộp luồng”. Nó có thể bị diễn giải thành “chạy lệnh ở chế độ nền (&) rồi tạo file (>)” — cho ra **kết quả sai mà không báo lỗi rõ ràng**. Vì vậy trong script

bắt đầu bằng `#!/bin/sh`, hãy **luôn** dùng dạng đầy đủ `> out.txt 2>&1` cho chắc.

Tự kiểm tra 4

- a) Lệnh `ls /thu-muc-ao` báo lỗi. Muốn lưu lỗi đó vào `bug.txt`, bạn dùng `>` hay `2>?`
b) lệnh `2>&1 > log.txt` có gộp được lỗi vào file không? Vì sao?
(Đáp án ở cuối chương.)

5 /dev/null — cái “hố đen” nuốt mọi thứ

`/dev/null` là một file đặc biệt của hệ điều hành: **mọi thứ ghi vào nó đều biến mất**, không lưu lại gì. Hãy coi nó là **thùng rác không đáy** hay **hố đen**. (Chính xác hơn, đây không phải file thường mà là một *thiết bị đặc biệt* — device file — nhưng bạn cứ hình dung nó như một file để dễ hiểu.)

Dùng để làm gì? Để **vứt bỏ** thứ bạn không quan tâm.

```
lenh 2> /dev/null      # vứt bỏ LỖI (chỉ giu ket qua tren man hinh)
lenh > /dev/null       # vứt bỏ KẾT QUẢ (chỉ con loi hien ra)
lenh > /dev/null 2>&1   # vứt bỏ TẤT CẢ (im lang hoan toan)
```

Ví dụ thực tế: bạn chạy một lệnh dò tìm trên cả ổ đĩa, nó in ra hàng trăm dòng “Permission denied” (không có quyền) gây nhiễu. Thêm `2> /dev/null` để **tắt tiếng ồn** đó đi, chỉ giữ lại kết quả thật.

Lưu ý

`/dev/null` chỉ “nuốt”, không “lưu”. Đừng dùng nó khi bạn còn cần xem lại nội dung sau này — lúc đó hãy ghi ra file thật.

6 Chuyển hướng đầu vào (stdin): <

Đến giờ ta toàn nói về đầu **ra**. Còn đầu **vào** thì sao?

Mặc định một lệnh nhận dữ liệu từ bàn phím. Dấu `<` bảo: “đừng chờ bàn phím, hãy **đọc nội dung từ file này** làm đầu vào”.

```
wc -l < ten-file.txt
```

Định nghĩa nhanh — wc

(word count): lệnh đếm. `wc -l` đếm **số dòng** (“l” = lines).

Khác biệt tinh tế:

- `wc -l ten-file.txt` — bạn **đưa tên file** cho `wc`, `wc` tự mở file. Kết quả in kèm tên file.

- `wc -l < ten-file.txt` — **shell** mở file và **rót nội dung** vào ống stdin của `wc`. `wc` không biết tên file, chỉ thấy dòng dữ liệu chảy vào. Kết quả chỉ có con số.

Với người mới, hai cách cho con số giống nhau. Nhưng hiểu `<` rất quan trọng vì nhiều lệnh **chỉ** nhận dữ liệu qua stdin, không nhận tên file.

7 Pipe | — nối các lệnh thành dây chuyền

Đây là “ngôi sao” của chương.

Pipe (cái ống, ký hiệu `|`) **nối stdout của lệnh bên trái thẳng vào stdin của lệnh bên phải**. Kết quả của lệnh trước trở thành đầu vào của lệnh sau, không cần file trung gian.

Tìm phím | ở đâu?

Trên đa số bàn phím kiểu Mỹ (US/ANSI), `|` nằm **cùng phím với** `\` (thường ở phía trên phím Enter), gõ bằng **Shift + **. Nếu bàn phím của bạn dùng layout khác (ISO, UK, quốc tế...), vị trí có thể khác — hãy tìm ký tự **gạch đứng |** in trên mặt phím, nó thường nằm gần phím Enter hoặc dùng Shift kèm phím dấu số đứng.

Hình dung một **dây chuyền sản xuất**: nguyên liệu chạy qua máy 1 → ra bán thành phẩm → chạy thẳng vào máy 2 → ... → ra thành phẩm cuối.

```
ls /usr/bin | wc -l
```

Đọc là: “liệt kê mọi thứ trong `/usr/bin` (ra một danh sách dài), rồi **rót thẳng** danh sách đó vào `wc -l` để đếm số dòng”. Kết quả: một con số = số mục trong thư mục đó.

7.1 Pipe khác Redirect như thế nào?

Đây là điểm cực kỳ hay nhầm:

	Nối với cái gì	Ví dụ
Redirect (<code>></code> , <code><</code>)	Nối lệnh với một FILE	<code>ls > out.txt</code>
Pipe (<code> </code>)	Nối lệnh với một LỆNH khác	<code>ls wc -l</code>

Câu thần chú: `>` **nói chuyện với file**, `|` **nói chuyện với lệnh**.

`ls > wc` sẽ tạo ra một **file** tên là “`wc`” (sai bét), còn `ls | wc` mới là **đưa danh sách cho lệnh wc xử lý** (đúng).

7.2 Vài lệnh hay đi cùng pipe

Chỉ giới thiệu đủ để ví dụ chạy được, ta sẽ học sâu ở chương sau:

- `wc` — đếm. `wc -l` đếm dòng, `wc -w` đếm từ.

- **grep tu-khoa** — **lọc**: chỉ giữ lại những dòng **có chứa** “tu-khoa” (khớp theo **chuỗi con liên mạch**, không phải “có đủ các chữ cái đó”).
- **sort** — sắp xếp các dòng theo thứ tự (mặc định A→Z).
- **head / tail** — lấy vài dòng **đầu / cuối** (bạn đã gặp ở chương trước).

Ví dụ ghép chuỗi:

```
ls /usr/bin | sort | head -5
```

“Liệt kê → sắp xếp A→Z → lấy 5 dòng đầu”. Mỗi | là một mắt xích trong dây chuyền.

Tự kiểm tra 7

- | nối lệnh với **file** hay với **lệnh khác**?
- grep loi nhat-ky.txt** làm gì?
(*Đáp án ở cuối chương.*)

Lưu ý nhỏ về thói quen tốt

Bạn sẽ thấy nhiều người viết `cat nhat-ky.txt | grep loi`. Cách này **chạy đúng**, nhưng thực ra thừa: **grep** tự đọc được file, nên viết gọn **grep loi nhat-ky.txt** là đủ (đây gọi vui là “dùng **cat** vô ích”). Trong chương này, khi nào ta dùng `cat ... | ...` thì chỉ là để **minh họa pipe** cho dễ **thấy dòng chảy dữ liệu**, chứ không phải cách gọn nhất.

8 THỰC HÀNH NGAY (Lab chính)

Thực hành ngay — Lab chính

Mở **Terminal** trên macOS (shell mặc định là `zsh`). Gõ **từng dòng**, nhấn Enter, quan sát. Đừng copy cả khối — gõ tay giúp nhớ lâu.

Trần an

Kết quả trên máy bạn **có thể hơi khác** (số đếm, thứ tự dòng, ngày giờ) tùy phiên bản macOS và nội dung máy. Điều đó **hoàn toàn bình thường** — bạn chỉ cần thấy **đúng kiểu** kết quả như mô tả.

8.1 Bước 1 — Tạo sân tập và vào đó

```
mkdir -p ~/linux-lab/ch4
cd ~/linux-lab/ch4
```

`mkdir -p` tạo thư mục (cờ `-p` để không báo lỗi nếu đã tồn tại). `cd` để bước vào.

Kết quả mong đợi: không có thông báo gì (im lặng = thành công). Gõ `pwd` sẽ thấy đường dẫn kết thúc bằng `/linux-lab/ch4`.

8.2 Bước 2 — stdout chảy vào file

```
echo "xin chao" > ra.txt
cat ra.txt
```

Kết quả mong đợi:

```
xin chao
```

`echo` lẽ ra in “xin chao” ra màn hình, nhưng `>` đã bẻ dòng chảy đó vào `ra.txt`. `cat` đọc lại file để xác nhận.

8.3 Bước 3 — stderr chảy vào file (lỗi KHÔNG hiện ra màn hình)

```
ls khong-co-file-nay 2> loi.txt
cat loi.txt
```

Kết quả mong đợi: Dòng `ls ...` không in lỗi ra màn hình (vì lỗi đã bị bẻ vào file). Khi `cat loi.txt`, bạn thấy đại loại:

```
ls: khong-co-file-nay: No such file or directory
```

Nội dung tiếng Anh chính xác có thể hơi khác chút tùy máy — miễn là có “No such file or directory” là đúng.

8.4 Bước 4 — Nuốt lỗi bằng `/dev/null`

```
ls khong-co-file-nay 2> /dev/null
```

Kết quả mong đợi: Không thấy gì cả — màn hình im lặng. Lỗi đã bị “hố đen” `/dev/null` nuốt sạch.

Vì sao trống trơn? Lệnh này vốn không có kết quả thật (nó chỉ sinh ra lỗi, không in stdout nào). Sau khi vớt lỗi đi thì màn hình trống là đúng — khác với trường hợp một lệnh vừa có kết quả vừa có lỗi: lúc đó `2> /dev/null` chỉ bỏ phần lỗi và bạn vẫn thấy kết quả. (So với Bước 3: ở đó lỗi được giữ trong file; ở đây lỗi bị vớt bỏ hẳn.)

8.5 Bước 5 — Pipe để đếm

```
ls /usr/bin | wc -l
```

Kết quả mong đợi: Một con số (ví dụ khoảng vài trăm đến hơn một nghìn, tùy máy — chẳng hạn 924). Đây là số dòng `ls` in ra, tức số mục trong `/usr/bin` không tính các mục ẩn (tên bắt đầu bằng dấu chấm `.`). Con số trên máy bạn gần như chắc chắn khác — không sao, miễn là ra một số dương. (Đừng neo vào một con số cụ thể.)

8.6 Bước 6 — Pipe vào sort

```
printf "cam\nbui\ntao\n" | sort
```

`printf` in văn bản; `\n` là ký tự **xuống dòng**, nên ta tạo ra ba dòng: `cam / buoi / tao`. Pipe rót ba dòng đó vào `sort`.

Kết quả mong đợi (đã sắp A→Z):

```
bui
cam
tao
```

8.7 Bước 7 — Pipe vào grep để lọc

```
printf "cam\nbui\ntao\n" | grep ao
```

`grep ao` chỉ giữ những dòng **chứa** chuỗi “ao”.

Kết quả mong đợi:

```
tao
```

Chỉ “tao” chứa “ao”; “cam” và “bui” bị loại.

8.8 Bước 8 — Gộp kết quả và lỗi vào cùng một file

```
ls / > danhsach.txt 2>&1
cat danhsach.txt
```

Lệnh `ls /` (liệt kê thư mục gốc) chạy trơn tru nên thường không có lỗi; nhưng cú pháp `> danhsach.txt 2>&1` đảm bảo: **nếu** có lỗi, nó cũng vào chung file này.

Kết quả mong đợi: `cat danhsach.txt` in ra danh sách các thư mục gốc của macOS, ví dụ:

```
Applications
Library
System
Users
...
```

Danh sách cụ thể tùy máy — chắc chắn có `Applications`, `Library`, `System`, `Users`.

8.9 Bước 9 — Dọn dẹp

```
cd ~
rm -ri ~/linux-lab/ch4
```

Quay về thư mục nhà (`cd ~`) rồi xóa thư mục `ch4` cùng mọi thứ trong nó. Ở đây ta thêm cờ `-i` (“interactive”) để hệ thống **hỏi xác nhận từng mục** trước khi xóa — gõ `y` rồi Enter để đồng ý. Khi đã quen tay, bạn có thể dùng `rm -r` (không hỏi) cho nhanh.

Kết quả mong đợi: sau khi xác nhận xong, gõ `ls ~/linux-lab` sẽ không còn thấy `ch4`.

Cần trọng

`rm -r` (không có `-i`) xóa **không hỏi lại và không vào Thùng Rác**. Luôn nhìn kỹ đường dẫn trước khi nhấn Enter. **Tuyệt đối không** ghép `rm -r` với một biến hay dấu `*` khi bạn chưa thật chắc chắn — đó là cách mất dữ liệu nhanh nhất. Ở đây ta ghi rõ đường dẫn tường minh `~/linux-lab/ch4` để khỏi xóa nhầm.

9 Bảng tóm tắt ký hiệu (in ra dán bàn)

Ký hiệu	Nghĩa
<code>></code>	Ghi stdout ra file, ghi đè nội dung cũ.
<code>>></code>	Ghi stdout ra file, nối thêm vào cuối.
<code>2></code>	Ghi stderr (lỗi) ra file, ghi đè.
<code>2>&1</code>	Đưa stderr đi theo nơi stdout đang trở (gộp lỗi vào đầu ra). Nhớ: đặt sau <code>> file</code> .
<code>&></code>	Gộp cả stdout lẫn stderr vào file (lối viết gọn của <code>zsh/bash</code> ; không dùng trên <code>sh/dash</code>).
<code><</code>	Lấy nội dung file làm stdin (đầu vào) cho lệnh.
<code> </code>	Pipe: nối stdout lệnh trái vào stdin lệnh phải.
<code>/dev/null</code>	“Hố đen” — mọi thứ ghi vào đều biến mất. Dùng để vứt bỏ.

10 Góc Senior (vì sao, production, cạm bẫy)

Phần này hé lộ tư duy của người làm DevOps thực thụ. Chưa hiểu hết cũng không sao — đọc để “ngấm” dần.

Góc Senior #1 — Tách stderr là xương sống của log và automation

Trong hệ thống thật, một dịch vụ chạy 24/7 sẽ tuôn ra rất nhiều dòng. Người ta thường tách: **kết quả/đầu ra bình thường (stdout)** đẩy vào nơi xử lý hoặc lưu trữ; **lỗi (stderr)** đẩy ra một file log riêng để cảnh báo. Ví dụ điển hình:

```
./chuong-trinh > /var/log/app.out 2> /var/log/app.err
```

Nhờ tách kênh, đội vận hành chỉ cần “nhìn vào file `.err`” là biết có sự cố, không phải lọc giữa hàng triệu dòng bình thường.

Góc Senior #2 — Cạm bẫy thứ tự 2>&1 (gặp hoài, kể cả người có kinh nghiệm)

Như mục 4: `> file 2>&1` đúng, còn `2>&1 > file` thì lỗi vẫn rò ra màn hình. Đây là lỗi kinh điển làm “log thiếu mất phần lỗi” trong các script thực tế. Thần chú: **file trước, gộp sau.**

Góc Senior #3 — Mã thoát của pipe: \$? chỉ là của lệnh CUỐI

Định nghĩa nhanh — mã thoát (exit code) và \$?

sau mỗi lệnh, hệ thống lưu một con số báo lệnh thành công (0) hay thất bại (khác 0). Bạn xem nó bằng `echo $?`.

Với pipe `A | B`, biến `$?` mặc định **chỉ phản ánh kết quả của B** (lệnh cuối). Nếu A chết giữa chừng mà B vẫn chạy ổn, shell vẫn báo “thành công” — rất nguy hiểm trong script. Cách khắc phục (sơ lược, sẽ học kỹ sau):

```
set -o pipefail
```

Đặt dòng này đầu script để shell coi pipe là **thất bại nếu BẤT KỲ** mắt xích nào thất bại, không chỉ mắt cuối.

Lưu ý portable

`set -o pipefail` có trong **bash** và **zsh**, nhưng **không** có trong POSIX **sh** thuần (dash). Nếu script cần tính năng này, hãy khai báo `#!/bin/bash` (hoặc `#!/bin/zsh`) thay vì `#!/bin/sh`; trong bash bạn cũng có thể kiểm tra mảng `${PIPESTATUS[@]}` để biết mã thoát của từng mắt xích. Đừng giả định mọi `/bin/sh` đều hỗ trợ `pipefail`.

Góc Senior #4 — Mỗi vế của pipe chạy trong một tiến trình con (subprocess) riêng

Hai lệnh hai bên `|` chạy **đồng thời**, mỗi cái trong một tiến trình con riêng, dữ liệu chảy qua ống nối giữa chúng (lệnh bên phải đọc dữ liệu ngay khi lệnh bên trái xuất ra, và sẽ **chờ** nếu chưa có dữ liệu; lệnh bên trái cũng có thể bị chặn nếu ống đã đầy — nên gọi “đồng thời” chính xác hơn “song song độc lập”). Hệ quả thực tế: một biến bạn đặt **bên trong một vế pipe** có thể “không sống sót” ra ngoài.

Quan trọng — kết quả tùy shell, đừng học thuộc một quy tắc tuyệt đối

- Trong **bash**, mọi vế của pipe chạy trong tiến trình con, nên biến đặt trong **bất kỳ** vế nào (kể cả vế cuối) đều **không** sống sót ra ngoài. Ví dụ kinh điển:

```
x=0; printf 'a\nb\nc\n' | while read l; do x=$((x+1)); done; echo $x
```

bash in ra 0 (biến mất).

- Trong **zsh** (mặc định trên macOS của bạn), **vế CUỐI** của pipe chạy trong shell hiện tại, nên cùng đoạn trên zsh lại in ra 3 (biến **sống sót**).

Vì script production thường chạy bằng bash/sh, **đừng dựa vào** việc biến sống sót qua pipe. Điều cần nhớ: **mỗi vế pipe là một tiến trình riêng**, và kết quả cụ thể có thể khác nhau giữa các shell — nếu thử trên máy zsh thấy biến vẫn còn thì đừng ngạc nhiên.

Góc Senior #5 — /dev/null để “tắt ồn” có chủ đích

Vứt lỗi nhiều vào /dev/null rất hữu ích, nhưng hãy **vứt có chọn lọc**. Nếu lỡ làm lệnh `> /dev/null 2>&1` cho một lệnh đang lỗi, bạn sẽ **không bao giờ biết** vì sao nó hỏng. Quy tắc: tắt ồn thì được, nhưng đừng tắt luôn cả tín hiệu báo cháy.

Góc Senior #6 — “Đừng phân tích đầu ra của ls trong script”

`ls` được thiết kế cho **con người đọc**, không phải cho máy đọc. Tên file có dấu cách, ký tự lạ, xuống dòng... sẽ làm `ls | ...` vỡ trận. Người có nghề dùng công cụ chuyên dụng (như `find`, hoặc vòng lặp `glob`) khi cần xử lý danh sách file bằng máy. Lưu ý: các ví dụ `ls /usr/bin | wc -l` hay `ls /etc | wc -l` trong chương này dùng `ls` để **đếm cho con người xem nhanh** thì vẫn ổn; điều cần tránh là **dựa vào từng dòng output của ls** để xử lý tên file bên trong script. Tư duy cốt lõi: công cụ cho người xem $\neq$ công cụ cho máy xử lý.

11 Hiểu lầm thường gặp (đọc kỹ để khỏi sai)

- “Dùng `>` cũng lưu được lỗi.” → **SAI**. `>` chỉ bắt `stdout`. Lỗi đi qua `stderr` (ống 2), nên `>` để mặc kệ lỗi chạy ra màn hình. Muốn lưu lỗi phải dùng `2>`.
- “Pipe `|` và redirect `>` giống nhau.” → **SAI**. `|` nối lệnh với lệnh; `>` nối lệnh với file. `ls > wc` tạo file tên “wc”; `ls | wc` đưa dữ liệu cho lệnh `wc`.
- “`2>&1` đặt ở đâu cũng được.” → **SAI**. Thứ tự quyết định kết quả. `> file 2>&1` gộp được lỗi; `2>&1 > file` thì không. **File trước, gộp sau**.

- “/dev/null lưu trữ tạm dữ liệu.” → **SAI**. Nó nuốt và xóa mọi thứ, không giữ lại gì.
- “**grep oi** giữ mọi dòng có chữ o và chữ i.” → **SAI**. grep khớp chuỗi con liền mạch. “chuo*i*” (ch-u-o-i) có “o” và “i” đứng liền nên khớp “oi”; còn “xoai” (x-o-a-i) tuy có cả “o” và “i” nhưng chúng **không đứng liền nhau** nên **không** khớp.

12 Thẻ ghi nhớ (Flashcards)

Che vé phải, tự trả lời, rồi kiểm tra.

- **stdin / stdout / stderr là gì + số hiệu?** → đầu vào (0) / đầu ra (1) / lỗi (2).
- **> vs >>?** → ghi đè / nối thêm (đều là stdout).
- **Bắt lỗi ra file?** → 2> (ghi đè), 2>> (nối thêm).
- **Gộp lỗi vào chung đầu ra, đúng thứ tự?** → lệnh > out.txt 2>&1 (file trước, gộp sau).
- **Viết gọn gộp cả hai?** → &> (zsh/bash; không dùng trên sh/dash).
- **Vứt bỏ mọi thứ?** → cho vào /dev/null.
- **Im lặng hoàn toàn?** → lệnh > /dev/null 2>&1.
- **Đầu vào từ file?** → < (vd wc -l < f.txt).
- **Nối hai lệnh?** → pipe |.
- **Pipe vs redirect?** → | nối lệnh↔lệnh; > nối lệnh↔file.
- **Mã thoát của pipe?** → mặc định là của lệnh **cuối**; thêm set -o pipefail (bash/zsh) để bắt lỗi giữa chuỗi.
- **grep khớp kiểu gì?** → khớp chuỗi con liền mạch, không phải “có đủ các chữ cái”.

13 Kế hoạch ôn ngắt quãng (Spaced Repetition)

- **Sau 1 ngày:** Mở Terminal, không nhìn sách, làm lại Bước 2, 3, 5, 6 của Lab. Tự hỏi: dùng > hay 2> để bắt lỗi?
- **Sau 3 ngày:** Giải thích **bằng lời** (nói ra miệng) sự khác nhau giữa pipe và redirect, và vì sao 2>&1 phải đặt sau. Làm “Bài tập mở rộng” số 1–3.
- **Sau 1 tuần:** Làm toàn bộ Bài tập mở rộng không nhìn đáp án. Vẽ lại sơ đồ “ba cái ống” của một lệnh từ trí nhớ. Nếu còn vấp chỗ nào, đọc lại đúng mục đó.

14 Tóm tắt 1 trang

- Mỗi lệnh có **3 ống**: **stdin (0)** đầu vào, **stdout (1)** kết quả, **stderr (2)** lỗi. **stderr** tách riêng để kết quả không lẫn lỗi. (Mặc định cả ba cùng trỏ tới cửa sổ Terminal — `tty`.)
- **Chuyển hướng stdout**: `>` ghi đè, `>>` nối thêm (`1> = >`).
- **Chuyển hướng stderr**: `2>` ghi đè, `2>>` nối thêm. `>` **không** bắt được lỗi.
- **Gộp lỗi vào đầu ra**: `lenh > out.txt 2>&1` (file trước, gộp sau). Viết gọn: `&>` (`zsh/bash`, không dùng trên `sh`).
- `/dev/null`: hố đen để vớt bỏ. Im lặng hoàn toàn: `lenh > /dev/null 2>&1`.
- **Chuyển hướng stdin**: `<` rót file vào đầu vào (vd `wc -l < f.txt`).
- **Pipe `|`**: nối `stdout` lệnh trái $\rightarrow$ `stdin` lệnh phải, tạo dây chuyền (vd `ls | sort | head`). `grep` khớp chuỗi con liền mạch.
- **Pipe $\neq$ redirect**: `|` nối lệnh $\leftrightarrow$ lệnh; `>` nối lệnh $\leftrightarrow$ file.
- **Senior**: tách `stderr` cho log; coi chừng thứ tự `2>&1; $?` là của lệnh cuối (cần `pipefail`, có ở `bash/zsh`); mỗi vế pipe là tiến trình con (`bash`: biến không sống sót ở mọi vế; `zsh`: vế cuối còn sống); đừng phân tích `ls` trong script.

15 Bài tập mở rộng (CÓ đáp án)

Thử tự làm trước khi xem đáp án. Tạo lại sân tập nếu cần: `mkdir -p ~/linux-lab/ch4`
&& `cd ~/linux-lab/ch4`.

Bài 1. Tạo file `ghichu.txt` chứa đúng hai dòng: “hom nay” và “hoc bai”, dùng `>` rồi `>>`.

Bài 2. Liệt kê một thư mục không tồn tại, vớt bỏ lỗi để màn hình im lặng (không tạo file lỗi).

Bài 3. Đếm xem trong `/etc` có bao nhiêu mục (dùng pipe).

Bài 4. Cho danh sách “xoai / mít / oi / chuối”, in ra **chỉ những dòng chứa chuỗi “oi”**, đã sắp xếp A $\rightarrow$ Z. (Gợi ý: `printf` + pipe + `grep` + `sort`.) Hãy **dự đoán** kết quả trước khi chạy — chú ý “oi” phải là hai chữ đứng **liền nhau**.

Bài 5. Chạy một lệnh sai và lưu **cả kết quả lẫn lỗi** vào `tatca.txt` bằng cú pháp **đúng thứ tự**, sau đó bằng cú pháp gọn `&>`.

Bài 6 (suy luận). Vì sao `cat khong-ton-tai.txt 2>&1 > out.txt` lại **không** ghi được lỗi vào `out.txt`? Lỗi sẽ đi đâu?

15.1 Đáp án Bài tập mở rộng

Bài 1.

```
echo "hom nay" > ghichu.txt
echo "hoc bai" >> ghichu.txt
cat ghichu.txt
```


Dùng `>` cho dòng đầu (tạo mới), `>>` cho dòng sau (nối thêm). Nếu dùng `>` cả hai lần, dòng đầu sẽ bị **ghi đè mất**.

Bài 2.

```
ls /thu-muc-khong-co 2> /dev/null
```

Lỗi bị `/dev/null` nuốt, màn hình im lặng, và **không** tạo file nào.

Bài 3.

```
ls /etc | wc -l
```

Ra một con số (tùy máy). `ls` liệt kê, pipe rót vào `wc -l` để đếm dòng.

Bài 4.

```
printf "xoai\nmit\nnoi\nchuoi\n" | grep oi | sort
```

Kết quả (đã sắp A→Z):

```
chuoi
oi
```

Giải thích: `grep` khớp **chuỗi con liền mạch** “oi”.

- “oi” — bản thân là “oi”, khớp.
- “chuoi” (ch-u-**o-i**) — có “o” và “i” đứng **liền nhau** ở cuối, nên **khớp**.
- “xoai” (x-**o-a-i**) — tuy có cả chữ “o” và “i” nhưng chúng **không** đứng liền nhau (bị chữ “a” chen vào), nên **không** khớp.
- “mit” — không có “oi”, bị loại.

Sau khi `sort` (A→Z): “chuoi” đứng trước “oi”. Đây là điểm dễ nhầm và là bài học quan trọng: `grep` tìm **chuỗi con**, không phải “có chứa các chữ cái đó ở đâu cũng được”.

Bài 5.

```
ls /khong-co > tatca.txt 2>&1      # cách đầy đủ, dùng thu tu
ls /khong-co &> tatca.txt          # cách gọn (zsh/bash)
cat tatca.txt
```

Cả hai đều đưa kết quả lẫn lỗi vào `tatca.txt`. (Nhớ: `&>` chỉ dùng trên `zsh/bash`, không dùng trong script `#!/bin/sh`.)

Bài 6. Vì shell đọc **trái sang phải**. Khi gặp `2>&1`, stdout lúc đó **vẫn đang trở ra màn hình**, nên stderr cũng được gán ra **màn hình**. Mãi sau `> out.txt` mới đổi stdout sang file, nhưng stderr đã “chốt” ở màn hình. Kết quả: **lỗi hiện ra màn hình**, còn `out.txt` chỉ chứa stdout (ở đây rỗng vì lệnh chỉ sinh lỗi). Muốn đúng: `cat khong-ton-tai.txt > out.txt 2>&1`.

15.2 Đáp án phần “Tự kiểm tra”

Tự kiểm tra 2.

- a) stdin = 0, stdout = 1, stderr = 2.
- b) Mặc định stderr **cũng hiện ra màn hình, giống chỗ hiển thị** của stdout (cả hai cùng trở tới cửa sổ Terminal — tty), nhưng là **kênh tách biệt** — nên ta có thể chuyển hướng riêng (2>) mà không đụng tới stdout.

Tự kiểm tra 4.

- a) Dùng 2> (lỗi đi qua stderr, ống số 2). > sẽ không bắt được.
- b) **Không**. Vì 2>&1 đứng **trước** > log.txt, nên lúc gộp, stdout còn trở ra màn hình → stderr cũng ra màn hình. Phải viết lệnh > log.txt 2>&1.

Tự kiểm tra 7.

- a) | nối lệnh với **một lệnh khác** (không phải file).
- b) grep loi nhat-ky.txt đọc nội dung file nhat-ky.txt và **lọc** chỉ giữ lại những **dòng có chứa chuỗi “loi”**. (grep tự đọc file nên không cần cat; nếu muốn minh họa pipe, có thể viết cat nhat-ky.txt | grep loi — cùng kết quả nhưng dài hơn.)